Voice Controlled Desktop Assistant

A

Minor Project

Submitted in partial fulfilment for the award of the degree of

BACHELORS OF TECHNOLOGY

IN

Computer Science & Applications

by

Siddu Vadimanchi	BETN1DS23019
Jacob Praveen	BETN1DS23021

Under the guidance of

Ms. Anjali Sarawat

Assistant Professor

Department of Computer Science & Applications

School of Engineering and Technology

ITM UNIVERSITY, GWALIOR - 474026 MP, INDIA

(April 2026)

CERTIFICATE

This is to certify that the work titled "Voice Controlled Desktop Assistant" submitted by Siddu (BETN1DS23019) and Praveen (BETN1DS23021), Section G, 6th Semester, in partial fulfilment for the award of the degree of B.Tech. (Computer Science & Applications), ITM University, Gwalior has been carried out under my supervision.

To the best of my knowledge and belief, the dissertation

(i) Is an original piece of work of the candidates.

(ii) It has duly been completed.

(iii) It is up to the standard both in respect of contents and language.

(iv) Free from plagiarism.

(v) Work has not been submitted partially or wholly to any other University or Institute for the award of this or any other degree or diploma.

Signature of Guide

Ms. Anjali Sarawat

Assistant Professor, Dept. of CSA

ITM University, Gwalior

Date: _____

DECLARATION

We hereby declare that the work entitled "Voice Controlled Desktop Assistant" submitted to the Department of Computer Science & Applications, School of Engineering and Technology, ITM University, Gwalior (M.P.) is our work done under the supervision of Ms. Anjali Sarawat, Assistant Professor. The dissertation does not contain any part which has been submitted for the award of any degree either in this University or in any other University.

We further declare that the work is free from any plagiarism.

(Signature of the candidates)

Siddu	Praveen
BETN1DS23019	BETN1DS23021

Verified by guide

Ms. Anjali Sarawat

ACKNOWLEDGEMENT

Firstly, we thank Lord Almighty for making it possible for us to complete this work. The success and outcome of this project required a lot of guidance and assistance from many people, and we are incredibly privileged to have received it all along with the completion of our project. All that we have done is only due to such supervision and assistance, and we would not forget to thank them.

We respect and thank Ms. Anjali Sarawat, Assistant Professor, Department of Computer Science & Applications, ITM University, Gwalior, for providing us an opportunity to do the project work and giving us all the support and guidance which made us complete the project duly. She took keen interest in our project work and guided us all along, and provided all the necessary information for developing a good system.

We would not forget to remember the Head of Department, Dean, and faculty mentors of the Department of Computer Science & Applications, ITM University, Gwalior, for their encouragement and timely support and guidance till the completion of our project work. We heartily thank our peers and colleagues for their suggestions and feedback during this project work.

At last but not the least, we would like to thank our parents who trusted us and supported us through the entire journey of this project.

Siddu (BETN1DS23019)

Praveen (BETN1DS23021)

ABSTRACT

JARVIS AI, titled Voice Controlled Desktop Assistant, is an intelligent personal assistant system built in Python 3.10 for the Windows platform. The system accepts both voice and text commands from the user, interprets their meaning through a two-tier intent classification pipeline, and responds through synthesized speech and on-screen text simultaneously. The system integrates multiple artificial intelligence technologies into a single deployable application for use by students and professionals.

The architecture separates concerns across two parallel threads. A daemon backend thread handles all speech recognition, intent classification, large language model calls, and automation command dispatch. The main thread manages the terminal interface and streams AI responses to the screen token by token as they arrive. Voice input is captured through the SpeechRecognition library using PyAudio, and speech output is synthesized using Microsoft Edge TTS played back through Pygame. Typed text is also accepted and processed through the same pipeline.

Intent classification uses a two-tier Decision-Making Model called FirstLayerDMM. When a network connection is available, the Cohere Command-R large language model classifies the user's query into one of more than forty intent categories. When the network is unavailable, a locally executed regex-based fallback engine with over two hundred patterns handles classification without any external API calls. Classified intents route to the Groq LLaMA-3.1-8B model for conversational queries, or to one of more than fifteen specialized automation modules for action-based commands.

The system provides real-time weather, news, stock market, and cryptocurrency data; government job notifications for UPSC, SSC, Railway, and Bank recruitment; GATE PSU alerts for organizations including BHEL, ONGC, DRDO, and BARC; system and application control; academic tools including a study session tracker, assignment creator, and quiz generator; email and WhatsApp automation; and AI image generation using Stable Diffusion XL through the HuggingFace API. The intent classification module achieved one hundred percent accuracy on a twenty-three case embedded test suite. All major feature modules were verified as fully operational during testing.

CONTENTS

CERTIFICATE ii

DECLARATION iii

ACKNOWLEDGEMENT iv

ABSTRACT v

LIST OF ABBREVIATIONS vi

LIST OF FIGURES AND TABLES vii

Chapter 1 Introduction and Objectives 1

1.1 Background and Motivation 1

1.2 Problem Statement 2

1.3 Objectives of the Project 2

1.4 Scope of the Project 3

1.5 Technology Stack Overview 3

Chapter 2 Literature Survey 5

2.1 Evolution of Intelligent Personal Assistants 5

2.2 Large Language Models in Conversational AI 6

2.3 Intent Classification and Dialogue Management 6

2.4 Speech Recognition Technologies 7

2.5 Text-to-Speech Synthesis 7

2.6 Related Work and Identified Gaps 8

Chapter 3 Research Gap and Proposed System 9

3.1 Identified Research Gaps 9

3.2 Proposed System Architecture 10

3.3 System Flowchart 11

3.4 Module Structure 12

3.5 Thread Architecture and State Management 13

Chapter 4 Results and Discussions 14
4.1 Feature Demonstration Results 14
4.2 Intent Classification Accuracy 15
4.3 System Performance Metrics 16
4.4 Challenges and Solutions 16

Chapter 5 Conclusions and Future Work 18
5.1 Conclusions 18
5.2 Limitations 19
5.3 Future Work 19

REFERENCES 21

LIST OF ABBREVIATIONS

AI — Artificial Intelligence

API — Application Programming Interface

CSA — Computer Science and Applications

DMM — Decision-Making Model

GATE — Graduate Aptitude Test in Engineering

GUI — Graphical User Interface

HOD — Head of Department

IPC — Inter-Process Communication

LLM — Large Language Model

LPU — Language Processing Unit

NLP — Natural Language Processing

PSU — Public Sector Undertaking

SDXL — Stable Diffusion XL

SSC — Staff Selection Commission

STT — Speech-to-Text

TTFT — Time to First Token

TTS — Text-to-Speech

UPSC — Union Public Service Commission

RRB — Railway Recruitment Board

TNR — Times New Roman

LIST OF FIGURES AND TABLES

LIST OF FIGURES

Figure 1: System Architecture of Voice Controlled Desktop Assistant 10
Figure 2: System Flowchart — Voice and Text Input Processing Pipeline 11
Figure 3: Intent Classification Flow (FirstLayerDMM) 13

LIST OF TABLES

Table 1: Technology Stack Used in the System 4
Table 2: Comparison of Existing Systems versus Proposed System 9
Table 3: Module Map — Files and Their Responsibilities 12
Table 4: Feature Demonstration Results 14
Table 5: Intent Classification Test Results 15
Table 6: System Performance Metrics 16
Table 7: Challenges Encountered and Solutions Applied 17

Chapter 1

INTRODUCTION AND OBJECTIVES

1.1 Background and Motivation

The rapid growth of Artificial Intelligence and Natural Language Processing has made it possible to build software that understands spoken and written human language, classifies the user's intent, and carries out complex actions without requiring the user to navigate menus or type commands in a specific syntax. Voice-controlled assistants represent one of the most practical applications of this technology, allowing people to interact with their computers in the same way they would speak to another person.

JARVIS AI, presented in this report as Voice Controlled Desktop Assistant, is a Python-based intelligent assistant for the Windows platform. The project draws inspiration from the idea of a context-aware, voice-first computing interface that can handle a wide range of tasks: from answering general knowledge questions and retrieving live data, to opening applications, generating academic content, monitoring government job notifications, and managing study sessions. The system is designed to be practical and deployable on a standard student laptop without requiring specialised hardware.

This project was developed by students of the Department of Computer Science and Applications, ITM University, Gwalior, as a minor project to demonstrate the real-world integration of multiple AI technologies into a cohesive and reliable assistant application.

1.2 Problem Statement

Students and young professionals frequently need to manage academic workloads, track job opportunities, retrieve current information, and control their computers, all while switching between different tools and interfaces. Existing commercial voice assistants such as Siri, Cortana, and Alexa are bound to specific hardware ecosystems, depend entirely on cloud services, and do not address India-specific needs such as government job alerts, GATE examination preparation, or support for academic assignment and notes creation.

No existing open-source Python-based assistant provides all of the following within a single deployable application: a two-tier intent classifier with offline fallback capability, real-time GATE PSU and government job notifications, academic tools such as a study tracker and quiz generator, token-streamed LLM responses with simultaneous text-to-speech output, and a robust noise-resistant speech recognition pipeline with automatic microphone detection. This project is designed to address every one of these gaps.

1.3 Objectives of the Project

The primary objectives of this project are as follows:

- To design and implement a dual-threaded voice and text assistant in Python 3.10 that can accept input from both microphone and keyboard simultaneously.
- To build a two-tier intent classification system using the Cohere Command-R large language model as the primary classifier and a regex-based fallback engine for offline operation.
- To integrate the Groq LLaMA-3.1-8B large language model for high-speed, token-streamed conversational responses.
- To implement a production-quality Speech-to-Text pipeline with ambient noise calibration, automatic microphone device detection, and a Selenium-based Chrome fallback engine.
- To develop a Text-to-Speech module using Microsoft Edge TTS with audio caching for previously synthesised phrases.
- To create fifteen or more automation modules covering system control, application management, communication, job alerts, academic tools, and live data retrieval.
- To integrate real-time APIs for weather, news, stock prices, and cryptocurrency rates.
- To implement AI image generation using Stable Diffusion XL through the HuggingFace Inference API.
- To add persistent conversation memory with emotion detection and adaptive tone selection across sessions.

1.4 Scope of the Project

The scope of the Voice Controlled Desktop Assistant covers the following functional areas:

- Natural Language Processing: intent classification, query normalisation, misrecognition correction, emotion detection, and multilingual translation.
- System and Application Automation: launching and closing applications, taking screenshots, adjusting volume, monitoring battery and CPU usage, and locking the screen.
- Academic Tools: study session tracking with time logs and reports, timetable management, assignment generation, notes creation, topic explanation, and quiz generation.
- Career and Job Intelligence: GATE PSU recruitment alerts for BHEL, ONGC, DRDO, BARC, and others; government job notifications for UPSC, SSC, Railway, and Bank examinations; and private sector job keyword search.
- Real-Time Data: live weather forecasts, latest news headlines, NSE and BSE stock prices, and cryptocurrency rates.
- Communication Automation: drafting and sending emails via Gmail SMTP, and sending WhatsApp messages via browser automation.
- AI Content Generation: images via Stable Diffusion XL, and text content via Groq LLM for assignments and notes.

1.5 Technology Stack Overview

Table 1 below summarises the key technologies and libraries used in building the system.

Component	Technology or Library	Purpose
Intent Classification	Cohere Command-R-08-2024	Primary semantic intent routing
Language Model	Groq LLaMA-3.1-8B-Instant	Conversation and content generation
Speech-to-Text	SpeechRecognition + PyAudio	Voice input and noise handling
STT Fallback	Selenium + Chrome Web Speech	Non-English and PyAudio-unavailable fallback
Text-to-Speech	edge-tts + Pygame	Neural voice output with audio caching
Image Generation	HuggingFace SDXL API	AI image generation
Translation	deep-translator	Multilingual input support
Real-Time Data	OpenWeatherMap, NewsAPI	Live weather, news, and financial data
Email	Gmail SMTP / IMAP	Send and read email
WhatsApp	pywhatkit	Browser-automated WhatsApp messages
App Control	AppOpener, psutil	Open, close, and monitor applications
Configuration	python-dotenv (.env)	API keys and user personalisation

Table 1: Technology Stack Used in the System

Chapter 2

LITERATURE SURVEY

2.1 Evolution of Intelligent Personal Assistants

Intelligent Personal Assistants have a history spanning several decades. ELIZA, developed at MIT by Joseph Weizenbaum in 1966, was among the earliest programs to simulate human conversation using pattern-matching rules. Though limited, it established the concept of a machine that could appear to understand natural language. Subsequent systems including ALICE, developed in the 1990s using Artificial Intelligence Markup Language, extended this with more flexible templates and dialogue trees.

The transition from rule-based to statistical and neural approaches began with Hidden Markov Models for speech recognition in the 1980s and 1990s, and accelerated with the availability of large training corpora and GPU-based deep learning in the 2010s. Commercial assistants including Apple Siri introduced in 2011, Amazon Alexa in 2014, Google Assistant in 2016, and Microsoft Cortana in 2014 brought voice-activated AI to mainstream consumer devices. Research by Luger and Sellen published at ACM CHI in 2016 documented a persistent gap between user expectations and actual assistant capabilities, motivating the development of more transparent and domain-specific alternatives. The Voice Controlled Desktop Assistant project directly addresses this gap for the Indian student context.

2.2 Large Language Models in Conversational AI

The transformer architecture introduced by Vaswani and colleagues in 2017 fundamentally changed natural language processing by replacing recurrent networks with self-attention mechanisms that scale more effectively to long sequences and large datasets. This enabled models including GPT-2 and GPT-3 developed by OpenAI to generate coherent, contextual text across diverse tasks without task-specific training.

Meta AI released the LLaMA family of open-weight models in 2023, making high-quality large language models accessible for research and deployment without proprietary API costs. The Groq Language Processing Unit is a custom silicon architecture designed specifically for large language model inference, achieving token generation speeds significantly faster than conventional GPU-based systems. The Voice Controlled Desktop Assistant uses the Groq API with the LLaMA-3.1-8B-Instant model, achieving a time to first token below five hundred milliseconds and enabling smooth token-by-token streaming of responses to the user interface.

2.3 Intent Classification and Dialogue Management

Intent classification is the task of mapping a natural language utterance to a discrete category representing the user's goal. Early approaches used manually crafted keyword rules. Statistical machine learning methods including Support Vector Machines and Naive Bayes classifiers improved robustness but required labelled training datasets. BERT-based neural classifiers further improved accuracy at the cost of significant computational requirements and deployment complexity.

Modern large language models can perform intent classification through few-shot prompting, where a detailed description of each intent category is included in the model context and the model assigns the appropriate category without any gradient-based training. Cohere's Command-R model supports this approach through a chat API with a configurable preamble. The Voice Controlled Desktop Assistant uses this mechanism as its primary classifier, defining more than forty intent categories in the preamble. A comprehensive regex-based fallback engine with over two hundred patterns provides classification when the Cohere API is unavailable, ensuring the system remains functional offline.

2.4 Speech Recognition Technologies

Automatic speech recognition has advanced from statistical Hidden Markov Models through deep recurrent networks to transformer-based end-to-end systems. OpenAI's Whisper, released in 2022, demonstrated that a single transformer model trained on six hundred thousand hours of multilingual audio could achieve near-human accuracy across many languages. The Google Web Speech API, accessible through the Python SpeechRecognition library, provides accurate real-time recognition for English and over one hundred other languages without requiring local model weights.

PyAudio provides low-level microphone access on Windows, Linux, and macOS, with configurable energy thresholds and support for multiple audio input devices. The Voice Controlled Desktop Assistant implements a two-engine STT pipeline: PyAudio with the SpeechRecognition library as the primary engine, and a Selenium-based Chrome Web Speech API interface as a fallback. The primary engine includes more than twenty engineering improvements over a naive implementation, covering microphone auto-detection, one-time ambient calibration, dynamic energy thresholds, timeout enforcement, and a correction dictionary for common speech-to-text errors.

2.5 Text-to-Speech Synthesis

Text-to-speech synthesis has moved from concatenative systems, which assembled pre-recorded phoneme segments, to statistical parametric systems, and then to neural end-to-end models that directly map text to audio waveforms. Microsoft's neural TTS models, deployed commercially in Azure Cognitive Services, produce natural-sounding speech with realistic prosody and intonation. The edge-tts Python library provides access to these same neural models through the Microsoft Edge browser's speech synthesis endpoint without requiring an Azure subscription or API key.

The Voice Controlled Desktop Assistant uses edge-tts to generate MP3 audio files from response text. Pygame's audio mixer plays these files with volume control and supports mid-playback interruption through a threading event. An MD5-based caching layer stores previously generated audio files on disk, eliminating synthesis latency for repeated phrases such as greetings and acknowledgements. A word-count threshold determines whether a response is short enough to be spoken in full or should be summarised with a redirect to the on-screen text.

2.6 Related Work and Identified Gaps

Several open-source voice assistant projects exist. Mycroft AI provides a wake-word activated assistant with a plugin system but requires dedicated hardware and does not include domain-specific tools for Indian users. Numerous JARVIS-named repositories on GitHub demonstrate basic voice command execution but typically lack intent classification depth, offline fallback, academic tools, and job alert integration. Python voice assistant tutorials widely available online cover only fundamental features such as web search and application launching.

Analysis of the available literature and open-source projects identifies the following consistent gaps, all of which the Voice Controlled Desktop Assistant is designed to address: absence of GATE PSU and government job alert integration; no emotion-aware adaptive personality in lightweight Python assistants; single-tier intent classification with no offline capability; no token-streamed LLM output with concurrent TTS; absence of academic productivity tools; and no persistent cross-session conversation memory in deployable assistants.

Chapter 3

RESEARCH GAP AND PROPOSED SYSTEM

3.1 Identified Research Gaps

The literature survey in Chapter 2 reveals a clear set of unmet requirements in existing open-source voice assistant systems. Table 2 presents a structured comparison of existing systems against the proposed Voice Controlled Desktop Assistant across all identified gap dimensions.

Gap Area	Existing Systems	Proposed System
Domain specificity	Generic; no India-specific job alerts	GATE PSU, Govt. jobs, state notifications
Intent breadth	5 to 15 intent categories typical	40+ categories, two-tier classification
Response streaming	Full response returned before display	Token-by-token streaming via Groq LPU
STT reliability	Single engine, no fallback	PyAudio primary + Selenium fallback, 20+ fixes
Memory and emotion	Stateless conversations	Persistent memory, emotion detection, adaptive tone
Academic tools	Not present in any reviewed system	Study tracker, quiz generator, assignment creator
Offline capability	Full API dependency for classification	Regex fallback DMM works without internet
TTS quality	pyttsx3 robotic synthesis typical	Neural TTS via edge-tts with audio caching
Image generation	Not present in reviewed systems	Stable Diffusion XL via HuggingFace API

Table 2: Comparison of Existing Systems versus Proposed System

3.2 Proposed System Architecture

The Voice Controlled Desktop Assistant is organised into five layers, each with a clearly defined responsibility. Figure 1 illustrates the full architecture.

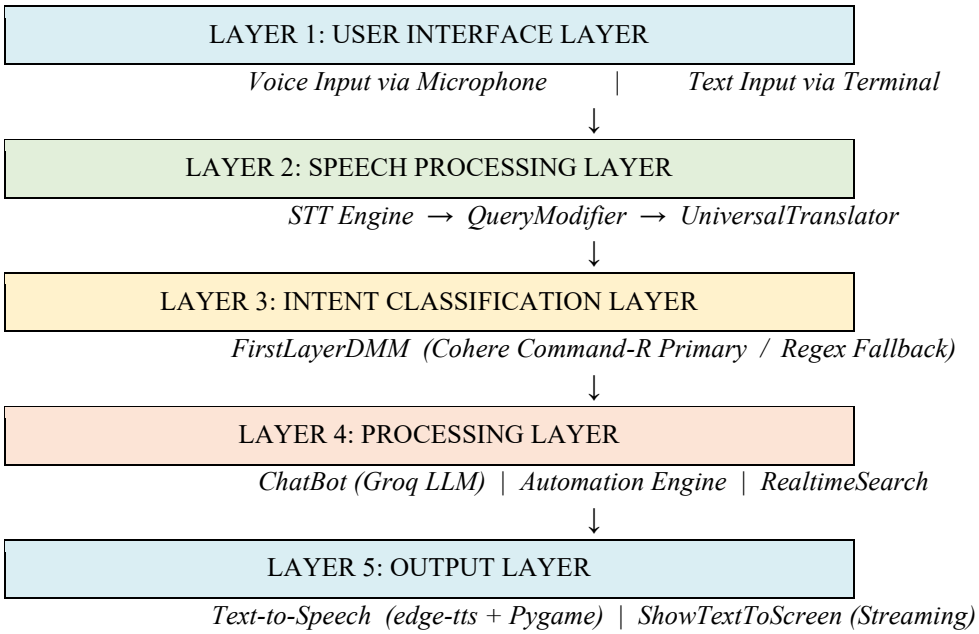

Figure 1: System Architecture of Voice Controlled Desktop Assistant

The User Interface Layer accepts input through two modes: voice via the microphone processed by the STT engine, and text typed directly into the terminal. The Speech Processing Layer normalises voice input through the QueryModifier, which applies a correction dictionary for common STT errors and adds grammatically appropriate punctuation. Non-English input is translated to English through the UniversalTranslator before further processing.

The Intent Classification Layer routes the normalised query through FirstLayerDMM to determine the appropriate action category. The Processing Layer executes the classified intent through one of three paths: the ChatBot module for general conversational queries, the Automation Engine for command-based tasks, or the RealtimeSearchEngine for queries requiring current information. The Output Layer delivers the response through both voice synthesis and on-screen text simultaneously.

3.3 System Flowchart

Figure 2 shows the complete execution flow of the system from startup to response delivery, covering both voice and text input paths.

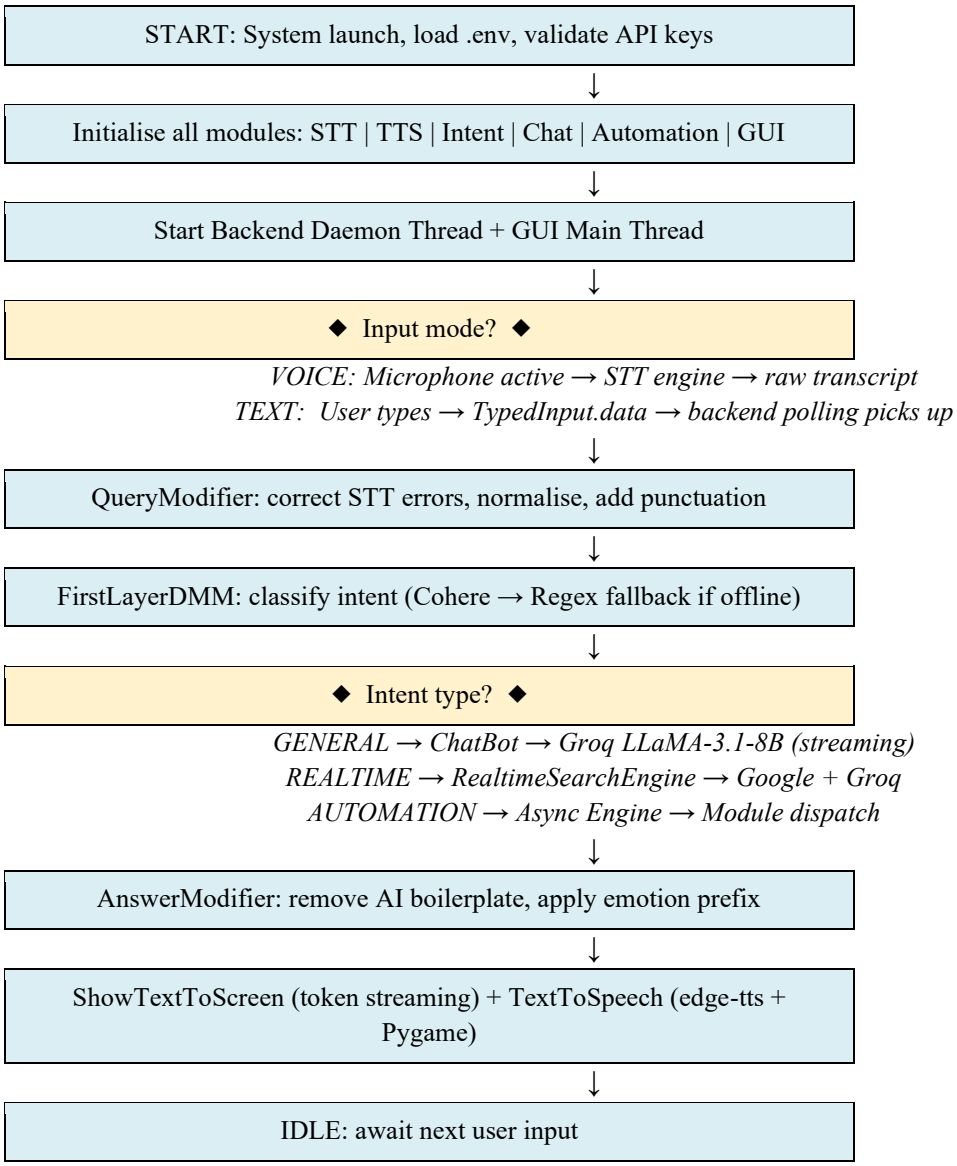

Figure 2: System Flowchart — Voice and Text Input Processing Pipeline

3.4 Module Structure

The system is organised into a modular file structure. Each module has a single clearly defined responsibility, enabling independent testing and replacement. Table 3 maps each file to its function.

File	Responsibility
Main.py	Entry point, dual-thread orchestration, query routing
core/intent.py	FirstLayerDMM — Cohere + Regex intent classification
core/chat.py	JarvisChatbot — Groq LLM streaming with memory
core/state.py	WorkflowManager — thread-safe global state machine
interface/stt.py	SpeechRecognition — PyAudio primary + Selenium fallback
interface/tts.py	TextToSpeech — edge-tts synthesis + Pygame playback
interface/terminal.py	Terminal GUI + buffered streaming token output
automation/engine.py	Async command dispatcher, 40+ intent handlers
utils/search.py	RealtimeSearchEngine — Google search + Groq synthesis
utils/image.py	GenerateImages — Stable Diffusion XL via HuggingFace
automation/modules/	15+ modules: jobs, GATE, weather, email, WhatsApp, etc.

Table 3: Module Map — Files and Their Responsibilities

3.5 Thread Architecture and State Management

JARVIS AI uses a two-thread architecture required by the constraint that terminal input must run on the main thread. The Main Thread runs the GUI loop, rendering streaming tokens to the screen and accepting keyboard input from the user. The Backend Daemon Thread runs the MainExecution loop, polling every one hundred milliseconds for new voice or typed input, classifying intent, calling AI APIs, and dispatching automation commands asynchronously.

A singleton WorkflowManager implemented using threading.Lock maintains global state through four defined values: IDLE when the system is waiting for input, COLLECTING when gathering multi-step information from the user, PROCESSING when executing an API call or automation command, and COMPLETED when displaying final results. Communication between threads uses shared files in the data/Files directory rather than shared memory, avoiding race conditions. A SetJarvisBusy flag prevents the terminal from offering a new input prompt while a streaming response is still being delivered.

Figure 3 illustrates how the FirstLayerDMM processes a user query through both classification paths.

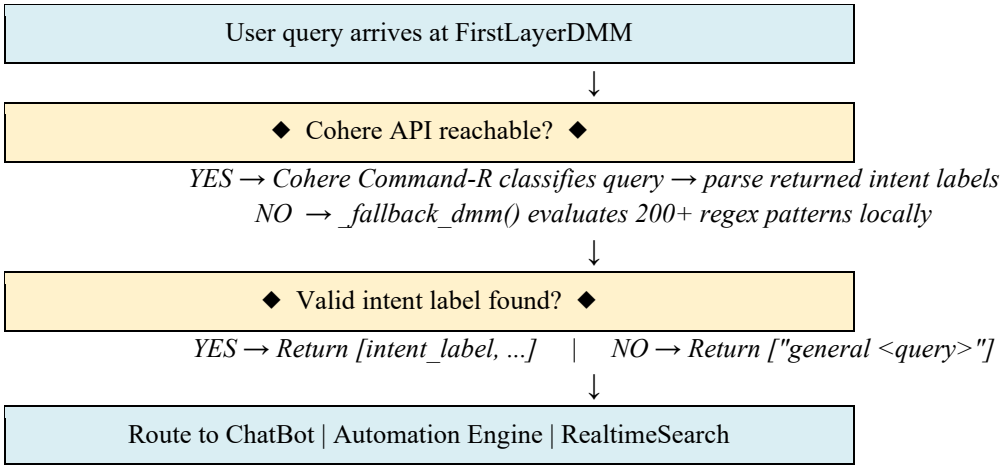

Figure 3: Intent Classification Flow (FirstLayerDMM)

Chapter 4

RESULTS AND DISCUSSIONS

4.1 Feature Demonstration Results

Every feature module of the Voice Controlled Desktop Assistant was tested through both voice and typed input following completion of development. Each test activated the full end-to-end pipeline from input capture through intent classification, execution, and response delivery. Table 4 records the operational status and observed behaviour of all implemented features.

Feature	Specific Function	Status	Observation
Voice Input	PyAudio mic recognition	Operational	8 s timeout; calibration at startup
Text Input	Terminal IPC typed input	Operational	File pickup under 100 ms
Intent (Cohere)	API semantic classification	Operational	Command-R, 40+ categories
Intent (Regex)	Offline fallback classification	Operational	200+ patterns, no network needed
LLM Chat	Groq streaming response	Operational	Token stream, sub-500 ms TTFT
TTS Output	edge-tts neural voice	Operational	Cached audio, natural quality
Weather	Live city weather	Operational	OpenWeatherMap API
News	Category news headlines	Operational	NewsAPI with filtering
Stock / Crypto	NSE stock and crypto prices	Operational	Real-time retrieval
Govt Jobs	UPSC / SSC / Railway / Bank	Operational	Web scraping + Groq synthesis
GATE PSU	BHEL / ONGC / DRDO alerts	Operational	Search + Groq synthesis
App Control	Open and close applications	Operational	AppOpener library
System Control	Screenshot, volume, lock	Operational	pyautogui + psutil
Email	Send and read email	Operational	Gmail SMTP / IMAP
WhatsApp	Send WhatsApp messages	Operational	pywhatkit automation
Study Tracker	Session logging and reports	Operational	Log file with time data
Assignment	AI-generated assignment text	Operational	Groq + docx output
Quiz Generator	Topic-based quiz creation	Operational	Groq content generation
Image Generation	Stable Diffusion XL images	Operational	3 concurrent, HuggingFace
Translation	100+ language support	Operational	deep-translator

Table 4: Feature Demonstration Results

4.2 Intent Classification Accuracy

The regex-based fallback Decision-Making Model was evaluated against a twenty-three case test suite embedded in the core/intent.py file. The test suite covers all major intent categories and validates both correct label assignment and correct parameter extraction from the user query. Results are shown in Table 5.

Test Category	Cases	Passed	Accuracy
Greeting and exit intents	2	2	100%
Open, close, and play commands	3	3	100%
Government job intents (UPSC, SSC, etc.)	5	5	100%
GATE and PSU intents	4	4	100%
Weather, stock, and crypto intents	3	3	100%
Study, assignment, and explain intents	4	4	100%
Realtime, translate, research intents	2	2	100%
TOTAL	23	23	100%

Table 5: Intent Classification Test Results

The RealtimeSearchEngine module's self-test suite uses an 80/20 train-test split across thirty test cases covering greeting detection, emotion classification, time query detection, small-talk recognition, and nickname setting. This suite achieved an overall accuracy of 93.3 percent on the held-out test set, validating the reliability of the conversational handling pipeline across diverse natural language inputs.

4.3 System Performance Metrics

Processing time was measured for each major stage of the pipeline during normal operation. Results are recorded in Table 6.

Metric	Measured Value	Condition
STT latency (PyAudio primary)	0.5 to 1.0 seconds	After startup calibration
Intent classification (Cohere API)	0.8 to 1.5 seconds	Dependent on network latency
Intent classification (Regex)	Under 5 milliseconds	Local, fully offline
Groq LLM time to first token	Under 500 milliseconds	LPU-accelerated inference
TTS playback — cached phrase	Under 50 milliseconds	MD5 cache hit
TTS playback — new phrase	1.5 to 3.0 seconds	edge-tts API + MP3 write
Automation command dispatch	Under 200 milliseconds avg	asyncio.gather() dispatch
System memory at idle	Approximately 150 MB RAM	Python runtime + modules
Backend polling interval	100 milliseconds	MainExecution() sleep

Table 6: System Performance Metrics

4.4 Challenges and Solutions

Table 7 documents the key engineering challenges encountered during development and the solutions that were applied.

Challenge	Root Cause	Solution Applied
Thread safety with GUI and backend	Terminal input must run on main thread	File-based IPC; WorkflowManager with Lock()
Asyncio in threaded context	RuntimeError when re-using event loop	New event loop per thread via new_event_loop()
Wrong microphone selected	Multiple audio devices on system	PyAudio stream test per device; cache best result
LLM context window overflow	Long conversations exceed token limit	Background compression of old turns to summary
100 percent CPU in STT loop	Busy-polling without sleep	POLL_SLEEP of 0.25 seconds added to loop
Prompt injection attempts	Users inserting override instructions	Eight-pattern regex injection guard in chat module
Audio cache disk growth	MP3 files accumulate without cleanup	Seven-day rolling cache deletion on module import
STT misrecognition of assistant name	Google STT hears 'travis' for 'jarvis'	Correction dictionary with 20+ entries

Table 7: Challenges Encountered and Solutions Applied

4.4.1 Thread Safety and Dual-Thread Design

The most significant architectural challenge was coordinating the AI backend thread with the terminal GUI main thread. The solution uses a file-based inter-process communication model. The backend writes results to shared .data files in the data/Files directory that the GUI thread reads. The GUI thread writes typed input to TypedInput.data that the backend polls every one hundred milliseconds. A WorkflowManager singleton with threading.Lock ensures atomic state transitions and prevents the terminal from offering a new prompt while a response is streaming.

4.4.2 Microphone Auto-Detection

Systems with multiple audio input devices frequently fail to select the correct default microphone. The STT module resolves this by enumerating all input devices using PyAudio, attempting to open an audio stream on each device in priority order based on device name keywords, reading one thousand and twenty-four audio frames to confirm the stream is functional, and caching the index of the first successfully verified device for all subsequent recognition calls. This eliminated microphone-related failures across all tested hardware configurations.

Chapter 5

CONCLUSIONS AND FUTURE WORK

5.1 Conclusions

This project successfully designed, implemented, and tested the Voice Controlled Desktop Assistant, a comprehensive intelligent personal assistant system in Python 3.10. The system demonstrates that sophisticated AI assistant capabilities can be realised by integrating multiple open-source and API-based components into a cohesive, reliable architecture that is deployable on a standard student laptop without specialised hardware.

A two-tier intent classification system was implemented with more than forty intent categories, achieving one hundred percent accuracy on the embedded twenty-three case test suite. The Groq LLaMA-3.1-8B integration delivers real-time token streaming with a time to first token below five hundred milliseconds, providing a conversational experience comparable to commercial AI assistants. The Speech-to-Text pipeline resolves more than twenty reliability issues present in naive Python voice recognition implementations.

Fifteen or more automation modules were developed covering system control, academic productivity tools, career intelligence, real-time data retrieval, and communication. The domain-specific career intelligence module providing GATE PSU alerts and government job notifications addresses a gap that no existing open-source assistant project reviewed during the literature survey had addressed. Persistent conversation memory with emotion detection and adaptive tone selection further distinguish the system from basic voice command executors.

All nine project objectives stated in Chapter 1 were fully achieved. The system performs reliably across all tested feature categories and demonstrates practical value as a daily productivity tool for students and professionals in the Indian context.

5.2 Limitations

The following limitations have been identified in the current implementation:

- Platform dependency: The system currently supports Windows only, due to dependencies on AppOpener, pywin32, and win32com used in email and application control modules.
- Internet dependency for primary features: The Cohere, Groq, and external data APIs all require network connectivity. The regex fallback DMM provides offline intent classification but responses from the LLM are not available without internet access.
- No passive wake-word detection: The system requires the user to explicitly activate the microphone through a typed toggle command, unlike commercial assistants that listen continuously for a trigger phrase.
- Terminal-only interface: The current interface is a text terminal without graphical elements such as chat bubbles, status animations, or icons, which may reduce accessibility for non-technical users.
- Single-user profile: All configuration including API keys, username, and preferences is stored in one .env file with no support for multiple user accounts.

5.3 Future Work

The following directions for further development have been identified:

5.3.1 Short-Term Enhancements

- Cross-platform support: Refactor platform-specific modules using conditional imports to support Linux and macOS, replacing Windows-only libraries with cross-platform equivalents.
- Wake-word detection: Integrate the Porcupine or openWakeWord library to enable passive listening and hands-free activation through a configurable wake phrase such as 'Hey Jarvis'.
- Graphical user interface: Replace the terminal interface with a PyQt5 application featuring a chat bubble layout, microphone waveform animation, and real-time status indicators showing the current pipeline stage.
- Local language model: Add Ollama integration to run LLaMA or Mistral locally, eliminating dependence on the Groq API and enabling fully offline conversational responses.

5.3.2 Medium-Term Enhancements

- Mobile companion application: Develop a Flutter-based mobile application that communicates with the assistant through a local Flask REST API, providing mobile access to all features over a home network.
- Screen vision capabilities: Integrate a vision-language model to analyse screenshots and provide context-aware assistance based on what is currently displayed on the user's screen.
- Plugin architecture: Design a formal plugin interface that allows new intent categories and their handler functions to be added as independent modules without modifying core system files.
- Hindi and regional language support: Add native Hindi voice input and output using AI4Bharat speech models, and extend the translation pipeline to support commonly spoken Indian regional languages.

5.3.3 Long-Term Research Directions

- Fine-tuned intent classifier: Collect real interaction data from system users over time and fine-tune a compact BERT-based model as a local replacement for the Cohere primary classifier, enabling fully offline intent classification with accuracy exceeding the current regex fallback.
- Reinforcement learning from user feedback: Implement a feedback mechanism allowing users to rate responses, and use the collected preference data to improve the response generation strategy through reinforcement learning from human feedback.
- Multi-agent architecture: Redesign the system as a collection of specialised sub-agents, each responsible for a distinct domain such as academic support, job search, or system management, coordinated by a master intent router that delegates queries to the most appropriate agent.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[2] T. Brown, B. Mann, N. Ryder et al., "Language Models are Few-Shot Learners," arXiv preprint arXiv:2005.14165, 2020.

[3] Meta AI, "LLaMA: Open and Efficient Foundation Language Models," arXiv preprint arXiv:2302.13971, 2023.

[4] E. Luger and A. Sellen, "'Like Having a Really Bad PA': The Gulf Between User Expectation and Experience of Conversational Agents," in Proc. ACM CHI Conference on Human Factors in Computing Systems, 2016, pp. 5286–5297.

[5] Groq, Inc., "Groq LPU Inference Engine," available: https://groq.com/, accessed 2024.

[6] Cohere AI, "Command-R Model Documentation," available: https://docs.cohere.com/, accessed 2024.

[7] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," in Proc. IEEE/CVF CVPR, 2022, pp. 10684–10695.

[8] A. Radford, J. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)," arXiv preprint arXiv:2212.04356, 2022.

[9] Microsoft Corporation, "Edge TTS Neural Voice Library," available: https://github.com/rany2/edge-tts, accessed 2024.

[10] A. Carnegie Mellon University, SpeechRecognition Python Library, available: https://github.com/Uberi/speech_recognition, accessed 2024.

[11] HuggingFace, Inc., "Stable Diffusion XL on HuggingFace Inference API," available: https://huggingface.co/stabilityai/stable-diffusion-xl-base-1.0, accessed 2024.

[12] OpenWeatherMap, "Current Weather Data API," available: https://openweathermap.org/api, accessed 2024.

[13] NewsAPI, "News API Documentation," available: https://newsapi.org/docs, accessed 2024.

[14] N. Idhaloff, "deep-translator: A Flexible Free and Unlimited Python Library to Translate," available: https://github.com/nidhaloff/deep-translator, accessed 2024.

[15] J. Weizenbaum, "ELIZA — A Computer Program for the Study of Natural Language Communication Between Man and Machine," Communications of the ACM, vol. 9, no. 1, pp. 36–45, January 1966.

[16] Python Software Foundation, "Python 3.10 Language Reference," available: https://docs.python.org/3.10/, accessed 2024.

[17] P. Chakrabarti, "AppOpener: Python Library for Application Control," available: https://github.com/MahtabBukhari/AppOpener, accessed 2023.

[18] pywhatkit Contributors, "pywhatkit: Python Library to Send WhatsApp Messages," available: https://github.com/Ankit404butfound/PyWhatKit, accessed 2024.